MULTIPLE REGRESSION & NEURAL NETWORKS

MULTIPLE REGRESSION

Regression analysis is a statistical methodology to estimate the relationship of a response variable (Y) to a set of predictor variables.

When there is just one predictor variable (X), and the response variable (Y) is continuous, we call it the **simple linear regression**.

When there are two or more predictor variables (X1, X2, …, Xk), and the response variable (Y) is continuous, we call it **multiple linear regression, or simply, multiple regression.**

History:

The earliest form of linear regression was the method of least squares, which was published by **Adrien-Marie Legendre** in 1805, and by **Johann Carl Friedrich Gauss** in 1809.

- The method was extended by **Sir Francis Galton** in the 19th century to describe a biological phenomenon.

- This work was extended by **Karl Pearson** and **Udny Yule** to a more general statistical context around 20th century.

MULTIPLE REGRESSION

Data: Inputs are continuous vectors of length k. Outputs are continuous variables.

$$\mathcal{D} = \{\boldsymbol{x}_i, y_i\} \; where \; \boldsymbol{x}_i = (x_{i1}, x_{i2}, \cdots, x_{ik})' \in \mathbb{R}^k \; and \; y_i \in R; \;\; i = 1,2, \cdots, n$$

Model: Here X_j represents the j[th] predictor and β_j quantifies the linear association between the given predictor and the response Y.

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \; \ldots + \beta_k X_{ik} + \; \epsilon_i; \quad i = 1,2, \cdots, n$$

Learning: The least squares method would find the parameters $\boldsymbol{\beta}$ that minimize the residual sum of squares (RSS), which is also called the sum of squared errors (SSE), as well as the quadratic loss**.**

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Prediction: Output is the estimated response value.

$$\hat{Y}_i = \; \widehat{\beta_0} + \widehat{\beta_1} X_{i1} + \; \ldots + \widehat{\beta_k} X_{ik} \; ; \quad i = 1,2, \cdots, n$$

MULTIPLE (LINEAR) REGRESSION

- In general, suppose we have k distinct predictors. Then the multiple regression model (also called the general linear model) takes the form

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ \ldots + \beta_k X_k + \ \epsilon$$

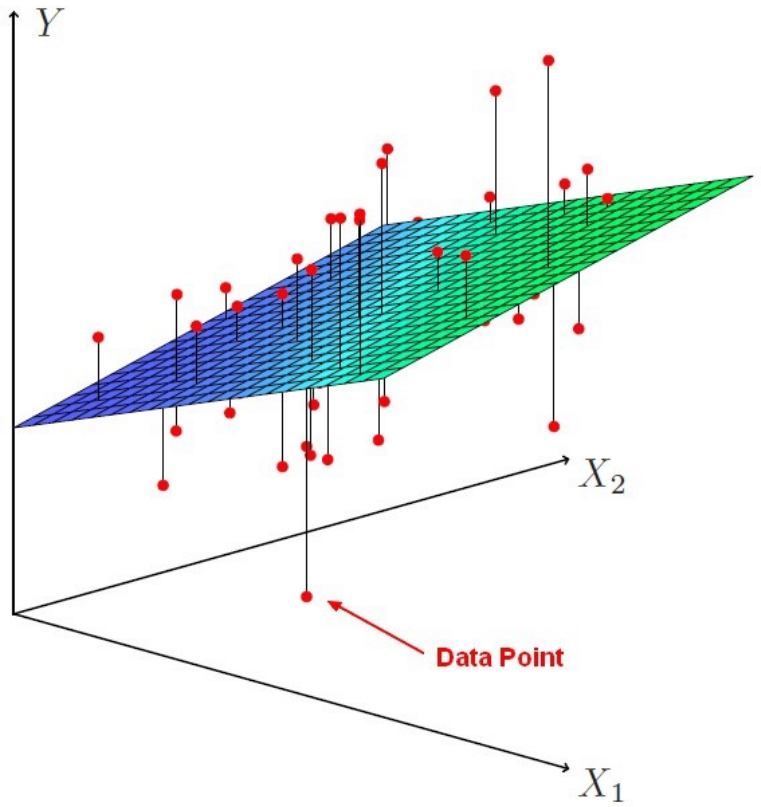
- Here X_j represents jth predictor and β_j quantifies the association between that variable and the response. We interpret β_j as the average effect on Y of a one unit increase in X_j, holding all other predictors fixed.
- Suppose we have n subjects, we would write the regression equation at the subject level as:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \ \ldots + \beta_k X_{ik} + \ \epsilon_i; \quad i = 1,2,\cdots,n$$

- The error terms $\epsilon_i, i = 1,2,\cdots,n$ are usually assumed to be independent.

ESTIMATING THE REGRESSION COEFFICIENTS

- The regression coefficients $\boldsymbol{\beta}$ are unknown and must be estimated. Given the estimates, we can make predictions using the formula
- $\hat{Y} = \widehat{\beta_0} + \widehat{\beta_1} X_1 + \ \ldots + \widehat{\beta_k} X_k$
- The parameters are usually estimated using the least squares approach. We choose coefficients $\boldsymbol{\beta}$ to minimize the sum of squared residuals ($e_i = y_i - \widehat{y_i}$), the RSS (also called the sum of squared error -- SSE):
- $RSS = SSE = \sum_{i=1}^{n} (y_i - \widehat{y_i})^2 = \sum_{i=1}^{n} e_i^2$
- The values of coefficients that minimize the RSS are the least squares estimators $\widehat{\boldsymbol{\beta}}$.



ESTIMATING THE REGRESSION COEFFICIENTS

- The least squares function is given by

$$L = \sum_{i=1}^{n} \epsilon_i^2 = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{k} \beta_j x_{ij} \right)^2$$

- The least squares estimators must satisfy

$$\left. \frac{\partial L}{\partial \beta_0} \right|_{\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_k} = -2 \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \sum_{j=1}^{k} \hat{\beta}_j x_{ij} \right) = 0$$

and

$$\left. \frac{\partial L}{\partial \beta_j} \right|_{\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_k} = -2 \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \sum_{j=1}^{k} \hat{\beta}_j x_{ij} \right) x_{ij} = 0 \quad j = 1, 2, \ldots, k$$

ESTIMATING THE REGRESSION COEFFICIENTS

- The **least squares normal Equations** are

$$
\begin{aligned}
n\hat{\beta}_0 + \hat{\beta}_1 \sum_{i=1}^{n} x_{i1} + \hat{\beta}_2 \sum_{i=1}^{n} x_{i2} + \cdots + \hat{\beta}_k \sum_{i=1}^{n} x_{ik} &= \sum_{i=1}^{n} y_i \\
\hat{\beta}_0 \sum_{i=1}^{n} x_{i1} + \hat{\beta}_1 \sum_{i=1}^{n} x_{i1}^2 + \hat{\beta}_2 \sum_{i=1}^{n} x_{i1}x_{i2} + \cdots + \hat{\beta}_k \sum_{i=1}^{n} x_{i1}x_{ik} &= \sum_{i=1}^{n} x_{i1}y_i \\
\vdots \qquad\qquad \vdots \qquad\qquad \vdots \qquad\qquad \vdots \qquad\qquad \vdots & \\
\hat{\beta}_0 \sum_{i=1}^{n} x_{ik} + \hat{\beta}_1 \sum_{i=1}^{n} x_{ik}x_{i1} + \hat{\beta}_2 \sum_{i=1}^{n} x_{ik}x_{i2} + \cdots + \hat{\beta}_k \sum_{i=1}^{n} x_{ik}^2 &= \sum_{i=1}^{n} x_{ik}y_i
\end{aligned}
$$

- The solution to the normal Equations are the **least squares estimators** of the regression coefficients.

MATRIX APPROACH TO MULTIPLE LINEAR REGRESSION

Suppose the model relating the regressors to the response is

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_k x_{ik} + \epsilon_i \qquad i = 1, 2, \ldots, n$$

In matrix notation this model can be written as

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

MATRIX APPROACH TO MULTIPLE LINEAR REGRESSION

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1k} \\ 1 & x_{21} & x_{22} & \cdots & x_{2k} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nk} \end{bmatrix} \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \end{bmatrix} \quad \text{and} \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

MATRIX APPROACH TO MULTIPLE LINEAR REGRESSION

We wish to find the vector of least squares estimators that minimizes:

$$L = \sum_{i=1}^{n} \epsilon_i^2 = \boldsymbol{\epsilon}'\boldsymbol{\epsilon} = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})'(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

The resulting least squares estimator for $\boldsymbol{\beta}$ is:

$$\widehat{\boldsymbol{\beta}} = (\boldsymbol{X}'\boldsymbol{X})^{-1}\boldsymbol{X}'\boldsymbol{y}$$

Note: When the random errors are independent and identically distributed normal random variables with mean 0 and variance σ^2, the maximum likelihood estimator for $\boldsymbol{\beta}$ *is the same as the above least squares estimator for* $\boldsymbol{\beta}$.

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} \qquad (12\text{-}15)$$

MATRIX APPROACH TO MULTIPLE LINEAR REGRESSION

The fitted regression model is:

$$\hat{y}_i = \widehat{\beta_0} + \widehat{\beta_1} X_{i1} + \ \ldots + \widehat{\beta_k} X_{ik}\,; \quad i = 1,2,\cdots,n$$

In matrix form, the fitted model is:

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\,\hat{\boldsymbol{\beta}}$$

The difference between the i[th] observed and fitted response value is the residual:

$$e_i = y_i - \hat{y}_i$$

Therefore, $RSS = SSE = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} e_i^2$

The $(n \times 1)$ vector of residuals is denoted by:

$$\mathbf{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$$

VARIABLE SELECTION

- It is possible that all of the predictors are associated with the response, but it is more often the case that the response is only related to a subset of the predictors.
- Three classical approaches for variable selection:
- 1. Forward selection
  - We begin with the null model - a model that contains an intercept but no predictors. We then add to the model the variable that results in the lowest RSS for the new 2-variable model. This approach continued until some stopping rule is satisfied.
- 2. Backward selection
  - We start with all variables in the model, and remove the variable with the largest p-value. This approach continued until some stopping rule is reached.
- 3. Stepwise variable selection
  - This is a combination of forward and backward selection.

VARIABLE SELECTION

- Best subset selection:

1. We select the best one-predictor model, best 2-predictor model, …, best p-predictor model, …, until the last one with all k-predictor.
2. We then choose the smallest p such as the increase in performance for p+1 etc is not significant.
3. This can be very computationally intensive as for best p-predictor model, there are $\binom{k}{p}$ models to choose from!

- For computational reasons, best subset selection cannot be applied with very large k. stepwise selection is a computationally efficient alternative to best subset selection.

LAB:
MULTIPLE REGRESSION

- In order to fit a multiple regression model using the least squares, we use the lm() function.
- The syntax lm(y~x1+x2+x3) is used to fit a model with three predictors, x1, x2 and x3.
- The summary function now outputs the regression coefficients for all the predictors.

```
library(MASS)
data("Boston")
lm.fit = lm(medv~lstat+age, data=Boston)
summary(lm.fit)
```

```
##
## Call:
## lm(formula = medv ~ lstat + age, data = Boston)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -15.981  -3.978  -1.283   1.968  23.158
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 33.22276    0.73085  45.458  < 2e-16 ***
## lstat       -1.03207    0.04819 -21.416  < 2e-16 ***
## age          0.03454    0.01223   2.826  0.00491 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.173 on 503 degrees of freedom
## Multiple R-squared:  0.5513, Adjusted R-squared:  0.5495
## F-statistic:   309 on 2 and 503 DF,  p-value: < 2.2e-16
```

'An Introduction to Statistical Learning with Applications in R' Chapter 3

LAB:
BEST SUBSET VARIABLE SELECTION METHODS

- The regsubsets() function performs best subset variable selection by identifying the best model that contains a given number of predictors, that is, best 1, best 2, best 3, etc. where best is quantified using RSS.
- An asterisk indicates that a given variable is included in the corresponding model.

```
library(ISLR)
library(leaps)
reg.fit = regsubsets(Salary~., Hitters)
summary(reg.fit)
```

```
## Selection Algorithm: exhaustive
##          AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns CRBI
## 1  ( 1 ) " "   " "  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 2  ( 1 ) " "   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 3  ( 1 ) " "   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 4  ( 1 ) " "   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 5  ( 1 ) "*"   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 6  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   " "    " "   " "    " "   "*"
## 7  ( 1 ) " "   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "   " "
## 8  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   " "    " "   "*"    "*"   " "
##          CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 ) " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 ) " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 ) " "    " "     " "       "*"     " "     " "    " "
## 4  ( 1 ) " "    " "     "*"       "*"     " "     " "    " "
## 5  ( 1 ) " "    " "     "*"       "*"     " "     " "    " "
## 6  ( 1 ) " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 ) " "    " "     "*"       "*"     " "     " "    " "
## 8  ( 1 ) "*"    " "     "*"       "*"     " "     " "    " "
```

'An Introduction to Statistical Learning with Applications in R' Chapter 6

LAB: SUBSET SELECTION METHODS

- We can also use the regsubsets() function to perform forward stepwise or backward stepwise selection, using the argument method='forward' or method ='backward'.
- The function stepAIC() from the MASS package is also frequently used.

```
library(ISLR)
library(leaps)
reg.fit = regsubsets(Salary~., Hitters, nvmax = 19, method = 'forward')
summary(reg.fit)
```

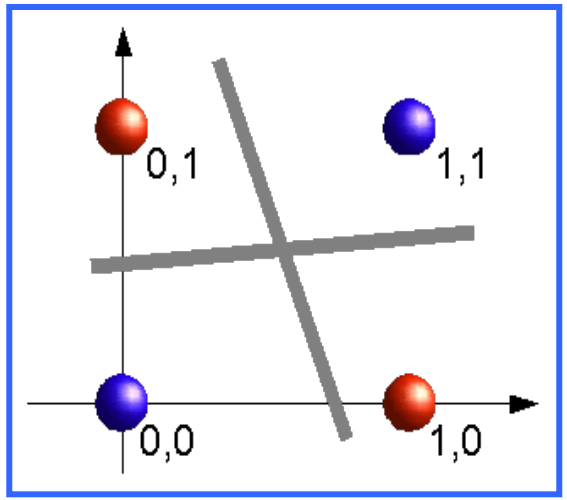
```
## 1 subsets of each size up to 19
## Selection Algorithm: forward
##          AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 2  ( 1 )  " "   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 3  ( 1 )  " "   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 4  ( 1 )  " "   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 5  ( 1 )  "*"   "*"  " "   " "  " " " "   " "   " "    " "   " "    " "   "*"
## 6  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   " "    " "   " "    " "   "*"
## 7  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   " "    " "   " "    " "   "*"
## 8  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   " "    " "   " "    "*"   "*"
## 9  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"   "*"
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"   "*"
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"   "*"
## 12  ( 1 ) "*"   "*"  " "   "*"  " " "*"   " "   "*"    " "   " "    "*"   "*"
## 13  ( 1 ) "*"   "*"  " "   "*"  " " "*"   " "   "*"    " "   " "    "*"   "*"
## 14  ( 1 ) "*"   "*"  "*"   "*"  " " "*"   " "   "*"    " "   " "    "*"   "*"
## 15  ( 1 ) "*"   "*"  "*"   "*"  " " "*"   " "   "*"    "*"   " "    "*"   "*"
## 16  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   " "   "*"    "*"   " "    "*"   "*"
## 17  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   " "   "*"    "*"   " "    "*"   "*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   " "    "*"   "*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"   "*"
##          CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       "*"     " "     " "    " "
## 4  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 8  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    "*"     "*"       "*"     "*"     " "    " "
## 12  ( 1 ) "*"    "*"     "*"       "*"     "*"     " "    " "
## 13  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    " "
## 14  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    " "
## 15  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    " "
## 16  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    " "
## 17  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

'An Introduction to Statistical Learning with Applications in R' Chapter 6

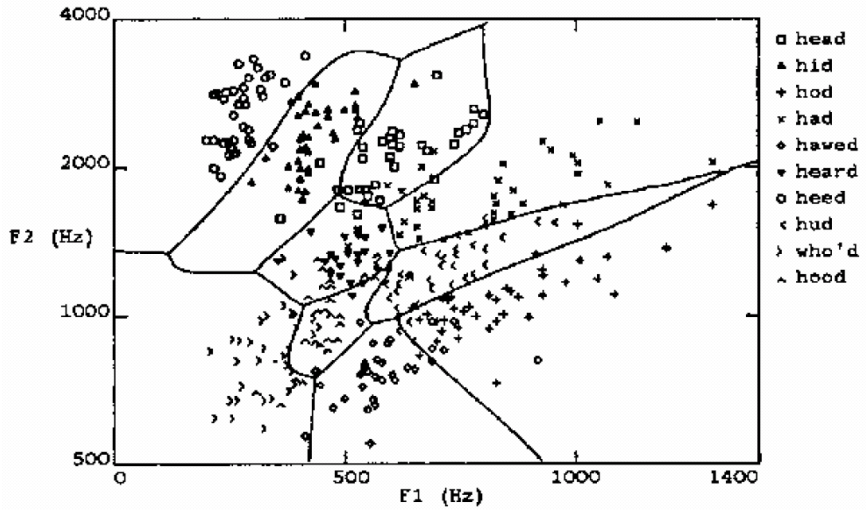
NEURAL NETWORKS

- **Learning highly non-linear functions**
- F: X --> Y
- X: (vector of) continuous and/ or discrete vars
- Y: (vector of) continuous and/ or discrete vars

The XOR gate

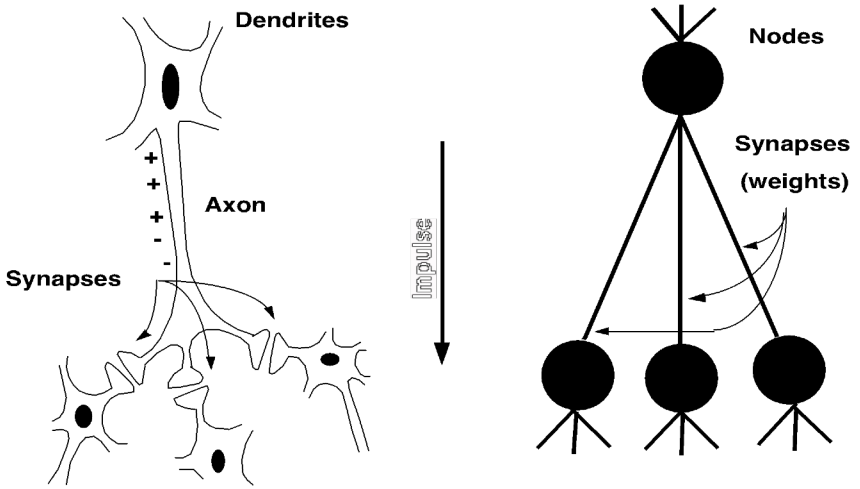


Speech recognition



CONNECTIONIST MODELS

- Consider humans:
  - Neuron switching time
    ~ 0.001 second
  - Number of neurons
    ~ 10^{10}
  - Connections per neuron
    ~ 10^{4-5}
  - Scene recognition time
    ~ 0.1 second
  - 100 inference steps doesn't seem like enough
    → much parallel computation
- Properties of artificial neural nets (ANN)
  - Many neuron-like threshold switching units
  - Many weighted interconnections among units
  - Highly parallel, distributed processes



A RECIPE FOR MACHINE LEARNING

- 1. Given training data: $\{\boldsymbol{x_i}, y_i\}$
- 2. Choose each of these:
  - Decision (activation) function $\hat{y} = f_\theta(\boldsymbol{x_i})$
  - Loss function $l(\hat{y}, y_i) \in \mathcal{R}$

3. Define goal: $\theta^* = argmin \sum l(f_\theta(\boldsymbol{x_i}), y_i)$

4. Train with SGD: (take small steps opposite the gradient)

$$\theta^{t+1} = \theta^t - \eta_t \nabla l(f_\theta(\boldsymbol{x_i}), y_i)$$

Goal:

1. Study the types of activation functions (decision functions)
2. Study the types of loss functions
3. Find the link between perceptron and multiple linear regression

Gradients

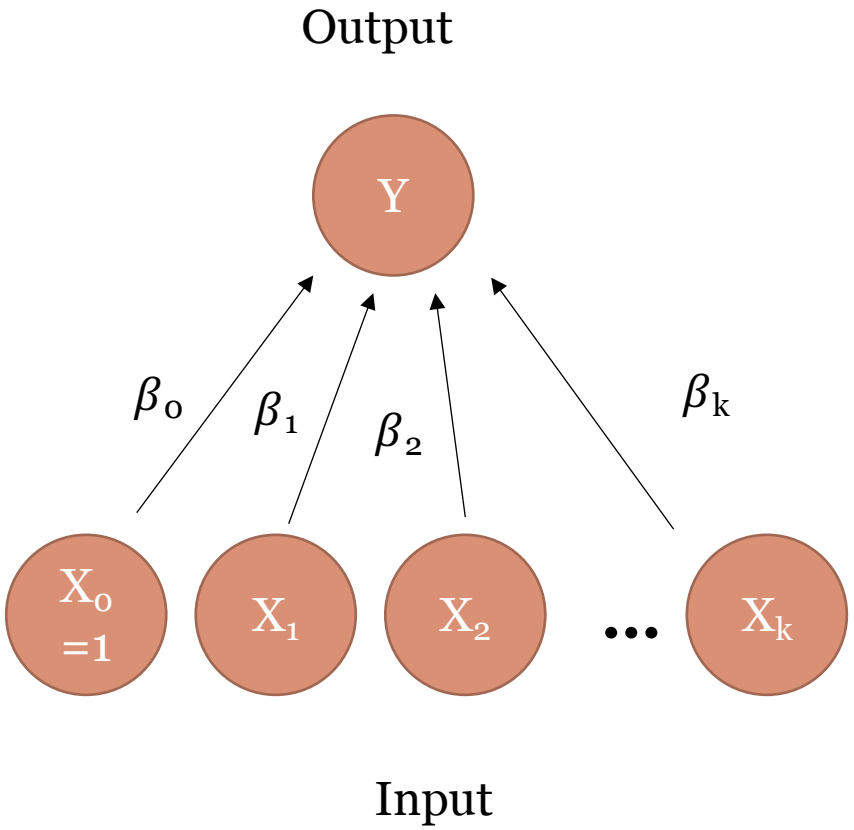
Backpropagation can compute this gradient. And it's a special case of a more general algorithm called reverse-mode automatic differentiation that can compute the gradient of any differentiable function effciently.

MULTIPLE REGRESSION AS A NEURAL NETWORK MODEL (WITH NO HIDDEN LAYER AND IDENTITY ACTIVATION FUNCTION)

- Here in multiple regression, the activation function is the identity link f**(a) = a**
- This is because:

$$E(Y|X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ \ldots + \beta_k X_k$$

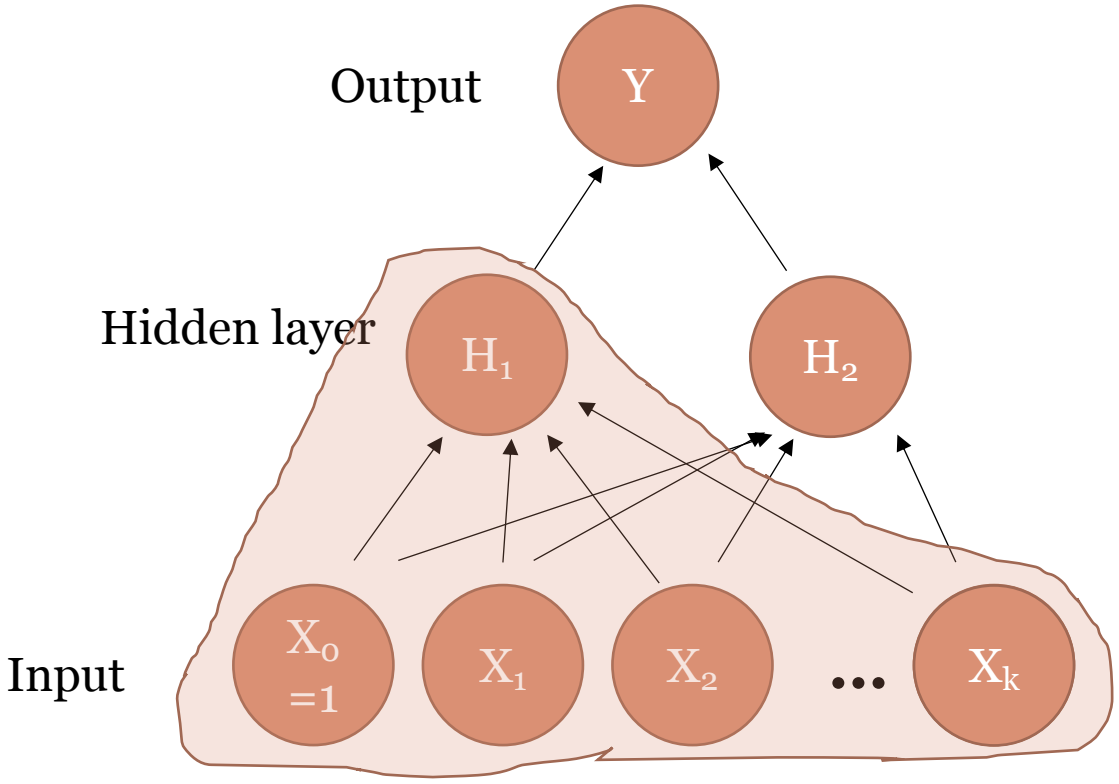
- Therefore it is sensible to set:
- $Y = f(\boldsymbol{\beta}^T \mathrm{X})$
- Then we look for $\boldsymbol{\beta}$ that will best predict the outout Y



Loss function: **SSE**
Activation function: **identity link**, which is also called the **linear activation function**

NEURAL NETWORK MODEL (WITH 1 HIDDEN LAYER AND IDENTITY ACTIVATION FUNCTION)

- The Neural Network model with or without hidden layers are both equal to the multiple regression model, if the identity activation function are applied at every layer.



NEURAL NETWORK (WITH IDENTITY ACTIVATION FUNCTION)

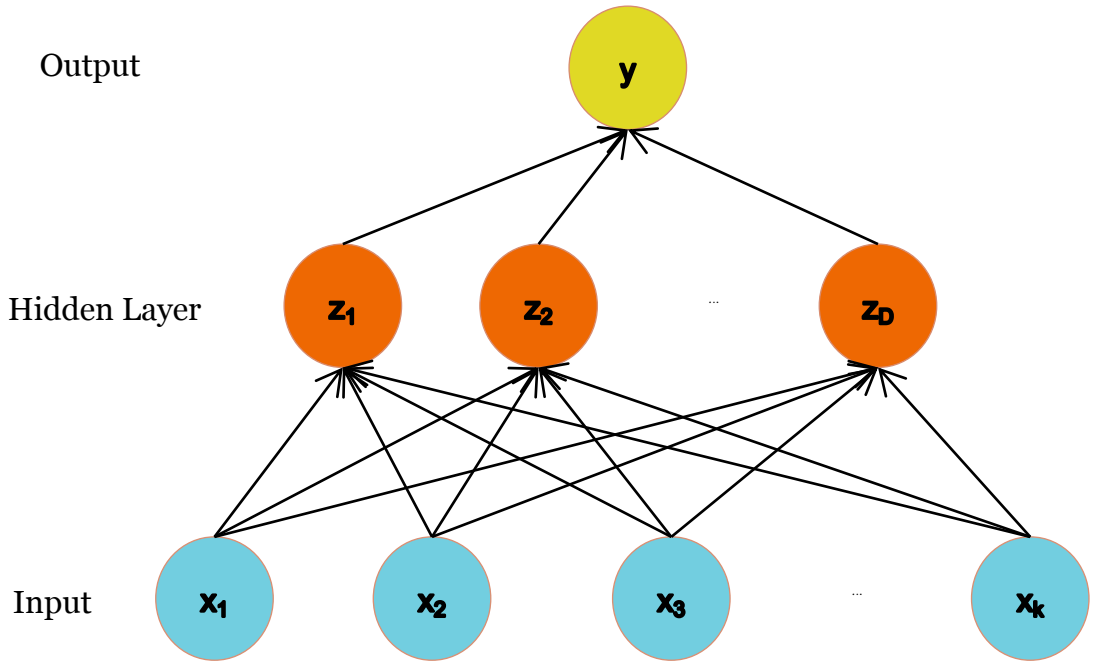
Output (linear):

$$y = \sum_{j=1}^{D} \beta_j Z_j$$

Hidden (linear):

$$Z_i = \sum_{j=1}^{k} \alpha_{ji} x_i$$

Input: given x_i



Y is still a linear combination of x_i.

ARCHITECTURES

- Even for a basic Neural Network, there are many design decisions to make:
- 1. # of hidden layers (depth)
- 2. # of neurons per hidden payer (width)
- 3. Type of activation function (non-linearity)
- 4. Form of objective function (loss function)

ACTIVATION FUNCTIONS

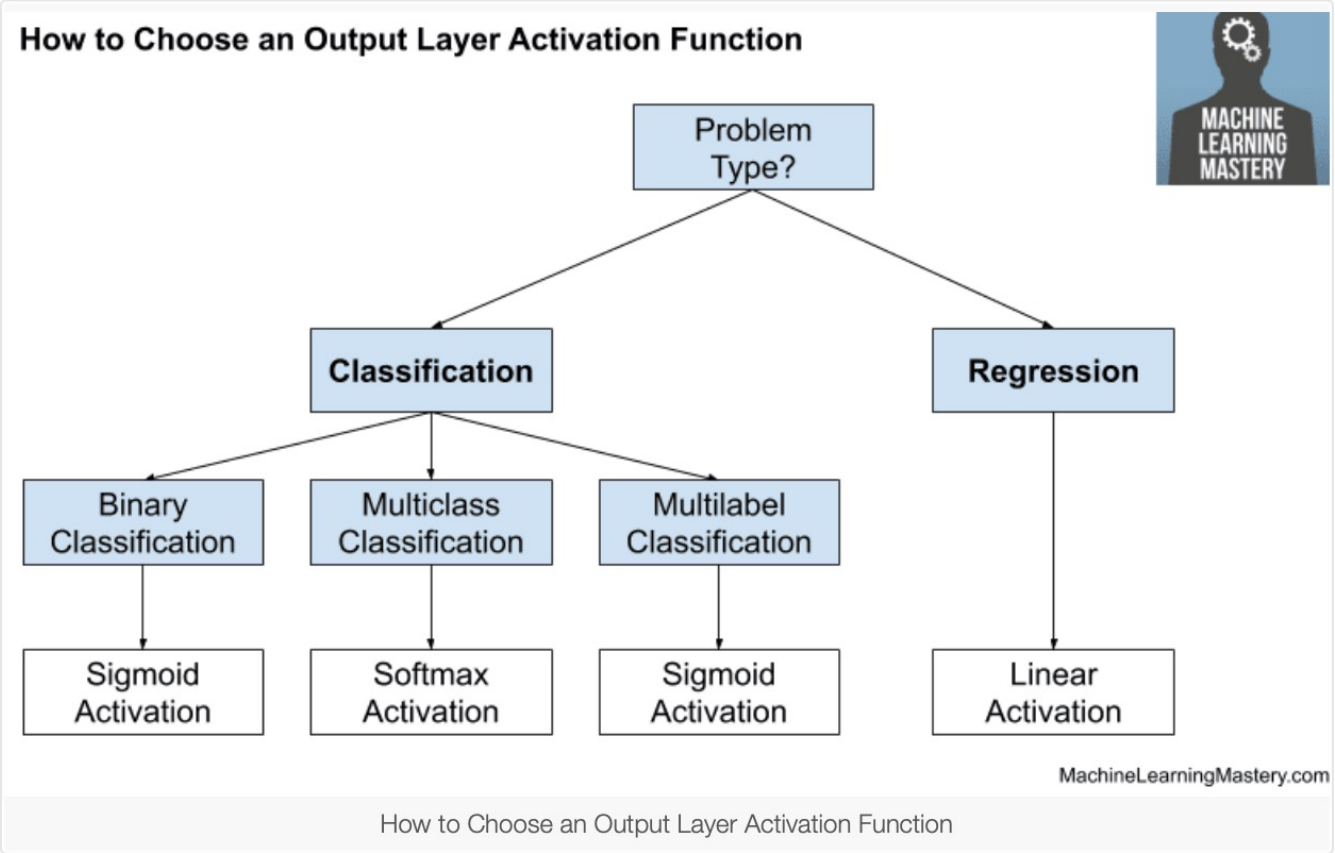
For the output layer:

- For continuous response, we can use the identity (linear) link function. (As we have compared to the multiple regression.)
- For binary response, we can use the sigmoid activation function. (As we have compared to the logistic regression)
- For other situations, we can use an arbitrary nonlinear function.

For the hidden layer(s):

- If we use the identify link function for the output layer, we should use at least one non-identity function in the hidden layer(s) to ensure we implement a truly nonlinear NN algorithm that is not identical to the multiple regression

ACTIVATION FUNCTIONS



How to Choose an Output Layer Activation Function

https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/

OBJECTIVE (LOSS) FUNCTIONS FOR NN

Typical loss functions are SSE and CE for both the regression and the classification tasks:

- SSE, also called RSS, also called the Quadratic Loss
- Cross-Entropy (i.e. negative log likelihood)

CROSS ENTROPY VS. QUADRATIC LOSS

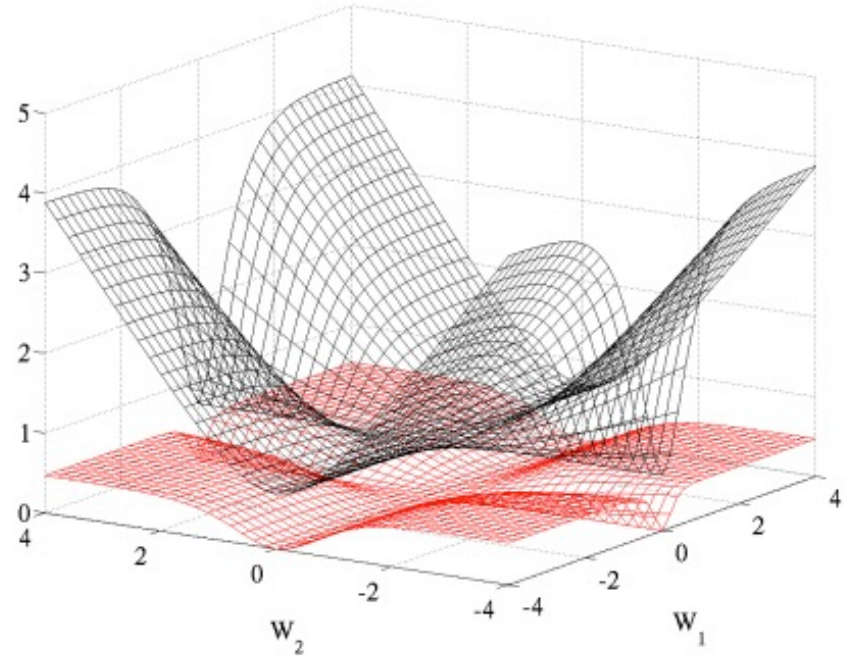
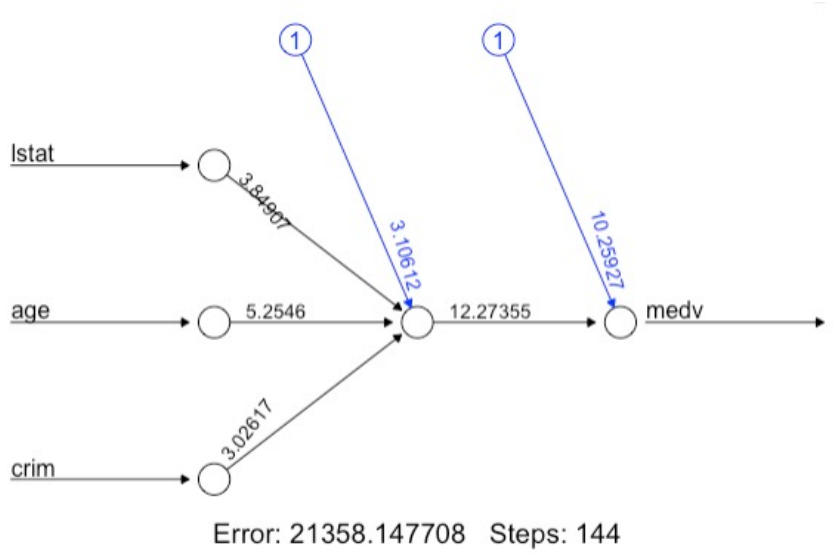


Figure 5: *Cross entropy (black, surface on top) and quadratic (red, bottom surface) cost as a function of two weights (one at each layer) of a network with two layers, W_1 respectively on the first layer and W_2 on the second, output layer.*

LAB: NEURAL NETWORK WITH LINEAR ACTIVATION FUNCTION

- In R, we can use neuralnet() function to generate neural network model.
- For more examples:
- https://datascienceplus.com/neuralnet-train-and-test-neural-networks-using-r/

```
library(neuralnet)
nn = neuralnet(medv ~ lstat+age+crim, data=Boston, linear.output = T)
plot(nn)
```



COMPARE MR AND NN WITH LINEAR ACTIVATION FUNCTION

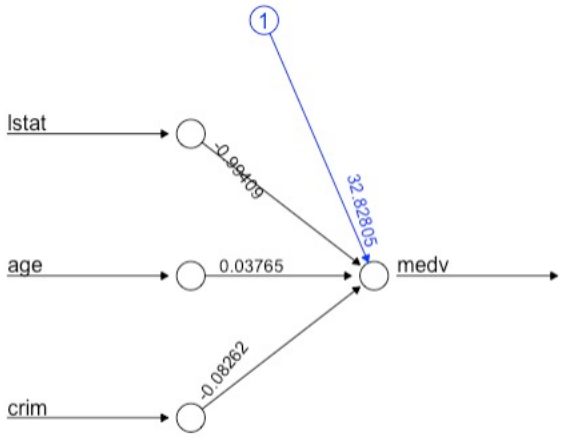
- Let's compare the result of MR and NN on the same data set:
- 1. Multiple regression:

```
lm(medv ~ lstat+age+crim, data=Boston)
```

```
##
## Call:
## lm(formula = medv ~ lstat + age + crim, data = Boston)
##
## Coefficients:
## (Intercept)        lstat          age         crim
##    32.82804     -0.99409      0.03765     -0.08262
```

- 2. Neural network model with identity activation function with no hidden layer:
  - The results of our models are similar! (coefficients)

```
nn = neuralnet(medv ~ lstat+age+crim, data=Boston, hidden = 0, linear.output = T)
plot(nn)
```



COMPARE MR AND NN WITH LINEAR ACTIVATION FUNCTION

- Neural network with identity activation function with one hidden layer:

```
library(keras)
library(mlbench)
library(dplyr)
library(magrittr)
library(neuralnet)

model = keras_model_sequential()
model %>% layer_dense(unit = 2, activation = 'linear', input_shape = c(3)) %>%
          layer_dense(unit = 1, activation = 'linear')
model %>% compile(loss = 'mse',  optimizer = optimizer_adam(learning_rate = .5), metrics = 'mse')
mymodel = model %>% fit(as.matrix(Boston[c('lstat', 'age', 'crim')]), Boston$medv, epochs = 500,
batch_size = 506)
```

```
[[1]]
<tf.Variable 'dense_1/kernel:0' shape=(3, 2) dtype=float32, numpy=
array([[-0.7109685 , -0.5018522 ],
       [ 0.15558992, -0.3301402 ],
       [-0.41135803,  0.91419846]], dtype=float32)>

[[2]]
<tf.Variable 'dense_1/bias:0' shape=(2,) dtype=float32, numpy=array([11.428112,  5.7855  ], dtype=float32)>

[[3]]
<tf.Variable 'dense/kernel:0' shape=(2, 1) dtype=float32, numpy=
array([[1.1095897],
       [0.4089007]], dtype=float32)>

[[4]]
<tf.Variable 'dense/bias:0' shape=(1,) dtype=float32, numpy=array([17.781855], dtype=float32)>
```

Intercept: MR: 32.82
NN: 11.42*1.10+5.78*0.40 + 17.78 = 32.65

Lstat: MR: -0.99
NN: -0.71*1.10-0.50*0.40 = -0.98

Age: MR 0.03
NN : 0.15*1.10-0.33*0.40 = 0.03

Crim: MR: -0.082
NN : -0.41*1.10 + 0.91*0.40 = -0.087

The results of the models are still similar.
* Small difference due to optimization method.

THANK YOU!

- Thanks to Ian (Weihao Wang) and Jason (Jiecheng Song) for their help compiling this lecture notes!!
- Our thanks also go to colleagues who made their data and resources available online.